In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

# ============================================================
# 2. Load the Auto MPG dataset
# ============================================================

url = (
    "https://raw.githubusercontent.com/"
    "mwaskom/seaborn-data/master/mpg.csv"
)

df = pd.read_csv(url)

print("First five rows of the dataset:")
print(df.head())

print("\nOriginal dataset shape:", df.shape)

# ============================================================
# 3. Preprocess the dataset
# ============================================================

# Select engine displacement and MPG
df = df[["displacement", "mpg"]].copy()

# Convert columns to numeric and handle invalid values
df["displacement"] = pd.to_numeric(
    df["displacement"], errors="coerce"
)

df["mpg"] = pd.to_numeric(
    df["mpg"], errors="coerce"
)

# Remove missing values
df.dropna(inplace=True)

print("\nDataset shape after preprocessing:", df.shape)

print("\nMissing values:")
print(df.isnull().sum())

# Features: engine displacement
# Target: miles per gallon
X = df[["displacement"]].values
y = df["mpg"].values

# ============================================================
# 4. Split the dataset into training and testing sets
# ============================================================
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# ============================================================
# 5. Implement Linear Regression
# ============================================================

linear_model = LinearRegression()

linear_model.fit(X_train, y_train)

# Predict MPG for test data
y_pred_linear = linear_model.predict(X_test)

# Evaluate Linear Regression
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print("\nLINEAR REGRESSION RESULTS")
print("-------------------------")
print("MSE:", round(mse_linear, 4))
print("R-squared:", round(r2_linear, 4))

# ============================================================
# 6. Implement Polynomial Regression of different degrees
# ============================================================

degrees = [2, 3, 4, 5]

polynomial_models = {}
results = []

# Store Linear Regression results
results.append({
    "Model": "Linear Regression",
    "Degree": 1,
    "MSE": mse_linear,
    "R-squared": r2_linear
})

for degree in degrees:

    print(f"\nTraining Polynomial Regression of degree {degree}...")

    # Create polynomial regression model
    model = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=False),
        LinearRegression()
    )

    # Train the model
    model.fit(X_train, y_train)
```

```
    # Predict test data
    y_pred = model.predict(X_test)

    # Evaluate the model
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    # Save model
    polynomial_models[degree] = model

    # Save results
    results.append({
        "Model": f"Polynomial Regression (Degree {degree})",
        "Degree": degree,
        "MSE": mse,
        "R-squared": r2
    })

    print("MSE:", round(mse, 4))
    print("R-squared:", round(r2, 4))

# ================================================================
# 7. Compare all models
# ================================================================

results_df = pd.DataFrame(results)

print("\n\nMODEL PERFORMANCE COMPARISON")
print("=" * 65)
print(results_df.round(4).to_string(index=False))

# ================================================================
# 8. Identify the best model based on test MSE
# ================================================================

best_index = results_df["MSE"].idxmin()
best_model_name = results_df.loc[best_index, "Model"]
best_mse = results_df.loc[best_index, "MSE"]
best_r2 = results_df.loc[best_index, "R-squared"]

print("\nBEST MODEL BASED ON TEST MSE")
print("----------------------------")
print("Model:", best_model_name)
print("MSE:", round(best_mse, 4))
print("R-squared:", round(best_r2, 4))

# ================================================================
# 9. Visualize Linear and Polynomial Regression fits
# ================================================================

# Generate evenly spaced displacement values for smooth curves
X_curve = np.linspace(
    X["displacement"].min() if isinstance(X, pd.DataFrame)
    else X.min(),
    X.max(),
    300
).reshape(-1, 1)

plt.figure(figsize=(11, 7))
```

```
# Plot actual data points
plt.scatter(
    X[:, 0],
    y,
    alpha=0.5,
    label="Actual data"
)

# Plot Linear Regression line
plt.plot(
    X_curve[:, 0],
    linear_model.predict(X_curve),
    linewidth=2,
    label="Linear Regression"
)

# Plot polynomial regression curves
for degree, model in polynomial_models.items():

    plt.plot(
        X_curve[:, 0],
        model.predict(X_curve),
        linewidth=2,
        label=f"Polynomial Degree {degree}"
    )

plt.xlabel("Engine Displacement")
plt.ylabel("Miles Per Gallon (MPG)")
plt.title("Linear vs Polynomial Regression on Auto MPG Dataset")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# ============================================================
# 10. Plot MSE comparison
# ============================================================

plt.figure(figsize=(10, 6))

plt.bar(
    results_df["Model"],
    results_df["MSE"]
)

plt.xlabel("Regression Model")
plt.ylabel("Mean Squared Error")
plt.title("MSE Comparison of Regression Models")
plt.xticks(rotation=30, ha="right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# ============================================================
# 11. Plot R-squared comparison
# ============================================================

plt.figure(figsize=(10, 6))
```

```
plt.bar(
    results_df["Model"],
    results_df["R-squared"]
)

plt.xlabel("Regression Model")
plt.ylabel("R-squared")
plt.title("R-squared Comparison of Regression Models")
plt.xticks(rotation=30, ha="right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# ============================================================
# 12. Conclusion
# ============================================================

print("\nCONCLUSION")
print("=" * 50)

print(
    "Linear Regression models the relationship using a straight line."
)

print(
    "Polynomial Regression uses powers of displacement to model "
    "a curved relationship."
)

print(
    "A lower MSE and a higher R-squared generally indicate "
    "better performance on the test dataset."
)

print(
    "The best model in this experiment based on test MSE is:",
    best_model_name
)
```

```
First five rows of the dataset:
    mpg  cylinders  displacement  horsepower  weight  acceleration  \
0  18.0          8         307.0       130.0    3504          12.0
1  15.0          8         350.0       165.0    3693          11.5
2  18.0          8         318.0       150.0    3436          11.0
3  16.0          8         304.0       150.0    3433          12.0
4  17.0          8         302.0       140.0    3449          10.5

   model_year origin                       name
0          70    usa  chevrolet chevelle malibu
1          70    usa          buick skylark 320
2          70    usa         plymouth satellite
3          70    usa              amc rebel sst
4          70    usa                ford torino

Original dataset shape: (398, 9)

Dataset shape after preprocessing: (398, 2)

Missing values:
displacement    0
mpg             0
dtype: int64

Training samples: 318
Testing samples: 80

LINEAR REGRESSION RESULTS
-------------------------
MSE: 18.1025
R-squared: 0.6633

Training Polynomial Regression of degree 2...
MSE: 15.1074
R-squared: 0.719

Training Polynomial Regression of degree 3...
MSE: 14.9436
R-squared: 0.7221

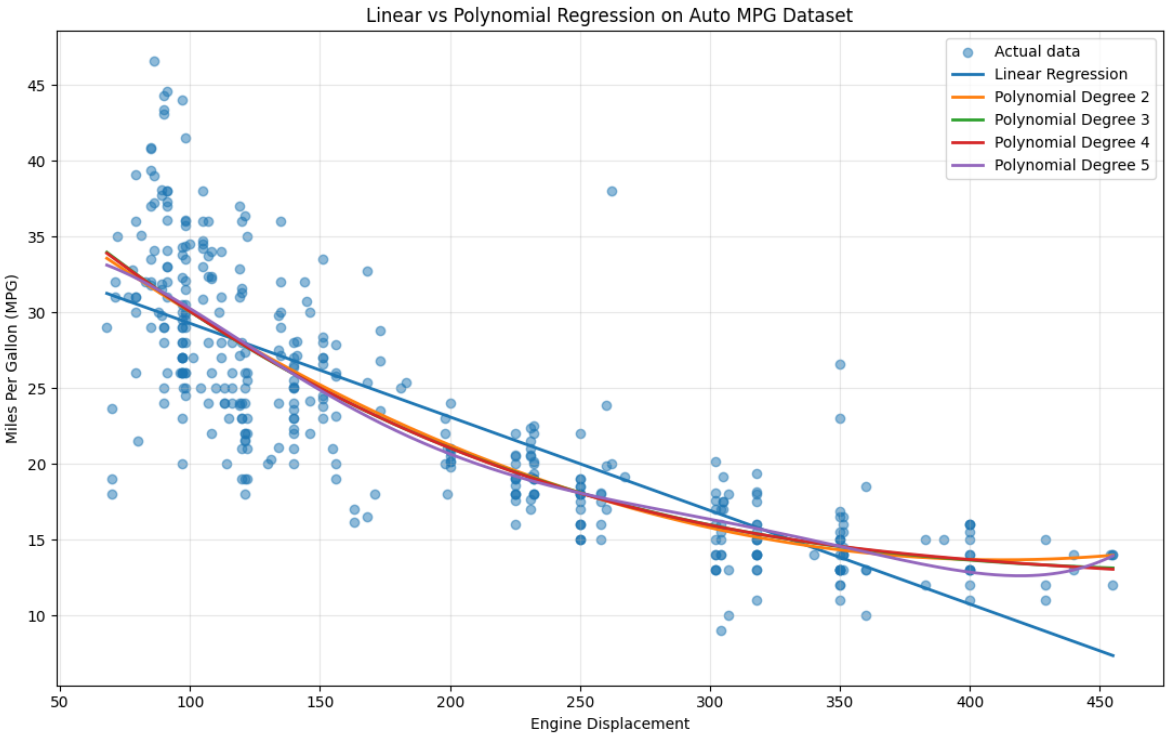
Training Polynomial Regression of degree 4...
MSE: 14.9615
R-squared: 0.7217

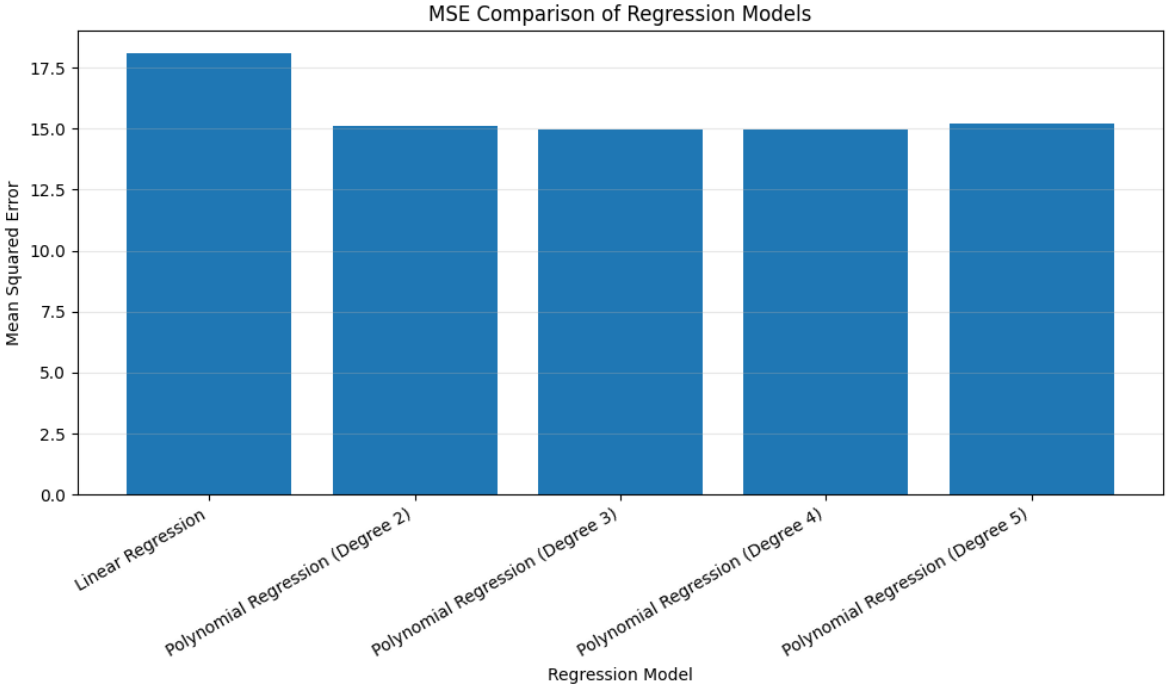
Training Polynomial Regression of degree 5...
MSE: 15.2309
R-squared: 0.7167

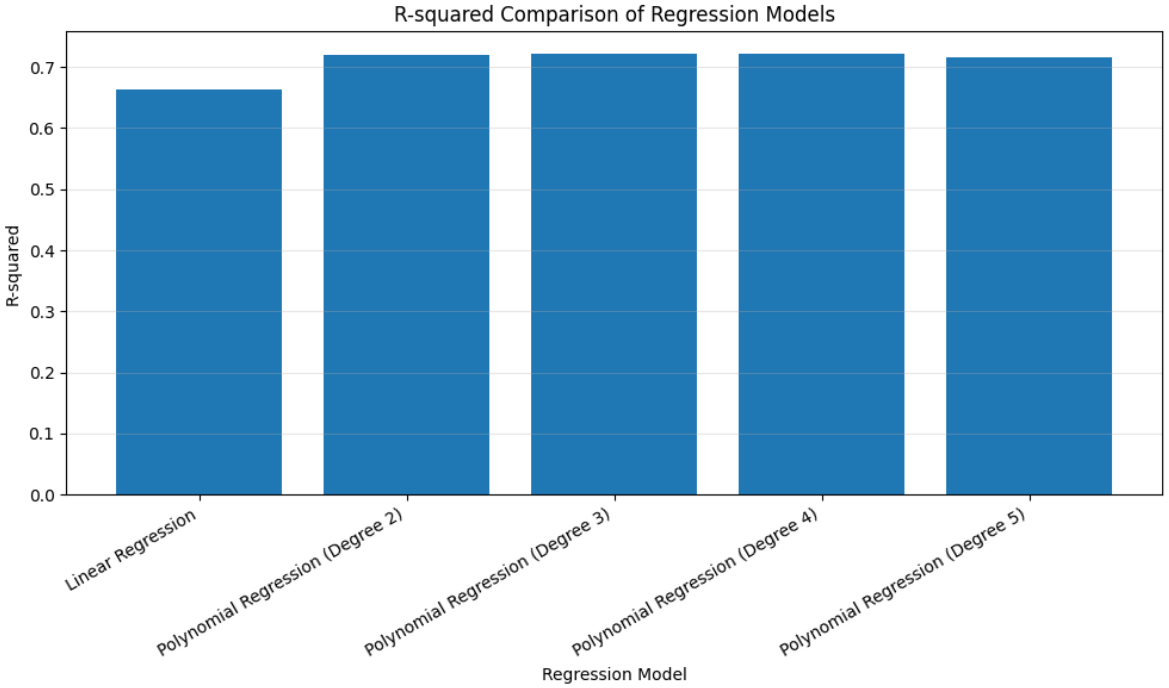

MODEL PERFORMANCE COMPARISON
======================================================================
                           Model  Degree     MSE  R-squared
               Linear Regression       1 18.1025     0.6633
Polynomial Regression (Degree 2)       2 15.1074     0.7190
Polynomial Regression (Degree 3)       3 14.9436     0.7221
Polynomial Regression (Degree 4)       4 14.9615     0.7217
Polynomial Regression (Degree 5)       5 15.2309     0.7167

BEST MODEL BASED ON TEST MSE
----------------------------
```

```
Model: Polynomial Regression (Degree 3)
MSE: 14.9436
R-squared: 0.7221
```








```
CONCLUSION
==================================================
Linear Regression models the relationship using a straight line.
Polynomial Regression uses powers of displacement to model a curved relati
onship.
A lower MSE and a higher R-squared generally indicate better performance o
n the test dataset.
The best model in this experiment based on test MSE is: Polynomial Regress
ion (Degree 3)
```

In [ ]: